第三章 神经网络

本章的核心目标是：

- 从感知机自然过渡到神经网络
- 理解激活函数的作用
- 掌握阶跃函数、Sigmoid 和 ReLU 的基本形式
- 学会用矩阵表示多层网络计算
- 理解输出层的设计
- 认识 softmax 的作用
- 对手写数字识别中的神经网络推理有初步理解

3.1 从感知机到神经网络

第二章中的感知机可以写成：

$$y = \begin{cases} 0 & (w_1x_1 + w_2x_2 \leq \theta) \\ 1 & (w_1x_1 + w_2x_2 > \theta) \end{cases}$$

感知机的特点是：

- 先做加权求和
- 再与阈值比较
- 输出只有 0 和 1

这种“超过阈值就输出 1，否则输出 0”的形式，本质上是一个**阶跃函数**。
感知机很重要，但它有一个明显限制：

> 输出变化是突变的，不是连续的。

而神经网络的关键进展就在于：

> 保留感知机的整体结构，但把阶跃函数替换成更平滑的激活函数。

也就是说：

- 感知机和神经网络在结构上很像
- 真正的区别主要在激活函数

3.2 激活函数

激活函数的作用是：

> 决定输入信号经过神经元后，如何被转换为输出信号。

如果没有激活函数，那么无论堆多少层网络，最后本质上仍然只是线性变换。
因此：

激活函数必须是非线性的。

3.2.1 阶跃函数

阶跃函数可以写成：

$$h(x) = \begin{cases} 0 & (x \leq 0) \\ 1 & (x > 0) \end{cases}$$

对应代码：

```
import numpy as np

def step_function(x):
    return np.array(x > 0, dtype=int)
```

它的含义是：

- 输入大于 0 时输出 1
- 否则输出 0

这就是感知机中最典型的激活方式。

3.2.2 Sigmoid 函数

神经网络中更常见的激活函数是 Sigmoid：

$$h(x) = \frac{1}{1 + e^{-x}}$$

它的特点：

- 输出在 0 到 1 之间
- 曲线平滑连续
- 输入越大，输出越接近 1
- 输入越小，输出越接近 0

代码实现：

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))
```

与阶跃函数相比，Sigmoid 更适合做连续优化。

3.2.3 为什么必须用非线性函数

假设某层输出为：

$$a = W_1 x + b_1$$

再经过下一层：

$$y = W_2 a + b_2$$

代入可得：

$$y = W_2(W_1 x + b_1) + b_2$$

整理后仍然只是一个线性变换。
这说明：

> 如果每一层都是线性的，那么多层网络和单层网络没有本质区别。

因此，神经网络必须引入非线性激活函数。

3.2.4 ReLU 函数

ReLU 的定义是：

$$h(x) = \begin{cases} 0 & (x \leq 0) \\ x & (x > 0) \end{cases}$$

代码实现：

```
import numpy as np

def relu(x):
    return np.maximum(0, x)
```

ReLU 的优点是：

- 形式简单
- 计算高效
- 在现代深度学习中使用非常广泛

3.3 多维数组与矩阵运算

为了高效实现神经网络，必须掌握 NumPy 的矩阵运算。

3.3.1 数组的形状

示例：

```
import numpy as np

A = np.array([1, 2, 3, 4])
print(A.ndim)
print(A.shape)
```

二维数组示例:

```
import numpy as np

B = np.array([[1, 2], [3, 4], [5, 6]])
print(B.ndim)
print(B.shape)
```

3.3.2 矩阵乘法

神经网络中最基本的计算之一就是矩阵乘法:

```
import numpy as np

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

print(np.dot(A, B))
```

矩阵乘法要求:

> 前一个矩阵的列数必须等于后一个矩阵的行数。

这和后面神经网络各层之间的维度匹配完全对应。

3.4 三层神经网络的实现

考虑一个简单的三层网络:

- 输入层
- 第 1 个隐藏层
- 第 2 个隐藏层
- 输出层

3.4.1 从输入层到第 1 隐藏层

设输入为:

```
X = np.array([1.0, 0.5])
```

设权重和偏置为:

```
W1 = np.array([[0.1, 0.3, 0.5],
               [0.2, 0.4, 0.6]])
b1 = np.array([0.1, 0.2, 0.3])
```

则:

```
A1 = np.dot(X, W1) + b1
Z1 = sigmoid(A1)
```

3.4.2 从第 1 隐藏层到第 2 隐藏层

```
W2 = np.array([[0.1, 0.4],
               [0.2, 0.5],
               [0.3, 0.6]])
b2 = np.array([0.1, 0.2])

A2 = np.dot(Z1, W2) + b2
Z2 = sigmoid(A2)
```

3.4.3 输出层

```
W3 = np.array([[0.1, 0.3],
               [0.2, 0.4]])
b3 = np.array([0.1, 0.2])

A3 = np.dot(Z2, W3) + b3
Y = A3
```

这里先把输出层函数设为恒等函数。

3.4.4 整体实现

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def identity_function(x):
    return x

def init_network():
    network = {}
    network["W1"] = np.array([[0.1, 0.3, 0.5],
                              [0.2, 0.4, 0.6]])
    network["b1"] = np.array([0.1, 0.2, 0.3])
    network["W2"] = np.array([[0.1, 0.4],
                              [0.2, 0.5],
                              [0.3, 0.6]])
    network["b2"] = np.array([0.1, 0.2])
    network["W3"] = np.array([[0.1, 0.3],
                              [0.2, 0.4]])
    network["b3"] = np.array([0.1, 0.2])
    return network

def forward(network, x):
    W1, W2, W3 = network["W1"], network["W2"], network["W3"]
```

```
    b1, b2, b3 = network["b1"], network["b2"], network["b3"]

    a1 = np.dot(x, W1) + b1
    z1 = sigmoid(a1)

    a2 = np.dot(z1, W2) + b2
    z2 = sigmoid(a2)

    a3 = np.dot(z2, W3) + b3
    y = identity_function(a3)

    return y
```

这段代码对应的就是神经网络前向传播的最基本过程。

3.5 输出层的设计

输出层使用什么激活函数，取决于任务类型。

3.5.1 回归问题

如果任务是预测连续数值，例如房价、温度、长度等，那么常用恒等函数：

$$f(x) = x$$

也就是不对输出再额外做非线性变换。

3.5.2 分类问题

如果任务是分类，尤其是多分类，常用 softmax 函数：

$$y_k = \frac{e^{a_k}}{\sum_{i=1}^{n} e^{a_i}}$$

它的作用是：

- 把输出转换成概率分布
- 每一项都在 0 到 1 之间
- 所有项加起来等于 1

代码实现：

```
import numpy as np

def softmax(a):
    c = np.max(a)
    exp_a = np.exp(a - c)
    sum_exp_a = np.sum(exp_a)
    return exp_a / sum_exp_a
```

这里减去最大值，是为了防止指数爆炸，保证数值稳定。

3.6 手写数字识别

神经网络一个非常经典的应用就是手写数字识别。

例如输入一张图像，输出它属于：

- 0
- 1
- 2
- ...
- 9

这实际上是一个 10 分类问题。

3.6.1 MNIST 数据集

MNIST 是经典的手写数字数据集，通常包含：

- 训练集
- 测试集
- 图像数据
- 对应标签

在这个任务中：

- 输入是图像像素
- 输出是 10 个类别的概率

3.6.2 推理流程

对于已经训练好的网络，推理的一般过程是：

1. 读入图像
2. 转成向量
3. 输入神经网络
4. 得到输出分数
5. 取最大值对应的类别

例如：

```
x = image_data
y = predict(network, x)
p = np.argmax(y)
```

其中：

- `predict` 表示前向传播
- `np.argmax` 表示取最大值下标

3.6.3 Batch 的概念

如果一次只预测一张图，效率会比较低。更高效的方法是一次处理一批图像，这就是 batch 处理。

Batch 的优点：

- 计算更高效
- 更适合矩阵并行
- 是后面深度学习训练和推理中的常见模式

3.7 本章小结

第三章的核心是：

- 从感知机过渡到神经网络
- 引入激活函数
- 用矩阵方式统一表示多层计算
- 根据任务类型选择输出层形式

需要特别掌握的内容包括：

- 阶跃函数、Sigmoid、ReLU
- 非线性函数的重要性
- 三层神经网络前向传播
- softmax 的作用及其数值稳定技巧
- 神经网络在分类任务中的基本使用方式

本章的重点在于建立“神经网络计算流程”的整体直觉。从第四章开始，学习重点将从“网络如何计算”转向“网络如何学习”。

3.8 附：本章核心代码

```
import numpy as np

def step_function(x):
    return np.array(x > 0, dtype=int)

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def relu(x):
    return np.maximum(0, x)

def identity_function(x):
    return x
```

```python
def softmax(a):
    c = np.max(a)
    exp_a = np.exp(a - c)
    sum_exp_a = np.sum(exp_a)
    return exp_a / sum_exp_a

def init_network():
    network = {}
    network["W1"] = np.array([[0.1, 0.3, 0.5],
                              [0.2, 0.4, 0.6]])
    network["b1"] = np.array([0.1, 0.2, 0.3])
    network["W2"] = np.array([[0.1, 0.4],
                              [0.2, 0.5],
                              [0.3, 0.6]])
    network["b2"] = np.array([0.1, 0.2])
    network["W3"] = np.array([[0.1, 0.3],
                              [0.2, 0.4]])
    network["b3"] = np.array([0.1, 0.2])
    return network

def forward(network, x):
    W1, W2, W3 = network["W1"], network["W2"], network["W3"]
    b1, b2, b3 = network["b1"], network["b2"], network["b3"]

    a1 = np.dot(x, W1) + b1
    z1 = sigmoid(a1)

    a2 = np.dot(z1, W2) + b2
    z2 = sigmoid(a2)

    a3 = np.dot(z2, W3) + b3
    y = identity_function(a3)

    return y
```